

Détection de vidéos générées par intelligence artificielle

par analyse de la cohérence géométrique 3D

Rapport de recherche

Encadrement scientifique : Emanuel Trabes, Carlos Valderrama

Année académique : 2025–2026

Date de remise : juillet 2026

Table des matières

Table des figures	iii
Table des expériences	iv
1 Remerciements	1
2 Note complémentaire	1
3 Introduction	1
3.1 Définition — Géométrie projective (Projective Geometry)	1
3.2 Problématique	2
3.3 Objectifs du projet	2
4 État de l’art et contexte scientifique	2
5 Démarche scientifique et méthodologie	3
5.1 Première approche : la cohérence photométrique	3
5.2 Discussion de la première méthode	4
5.3 Deuxième approche : la cohérence géométrique	5
5.4 Pour réaliser cette reprojection, trois informations sont nécessaires :	5
5.5 Estimation de la profondeur	6
5.6 Amélioration de la robustesse	8
5.7 Gestion de la validité des pixels	9
5.8 Intégration dans notre pipeline	10
5.9 Création du jeu de données	10
5.10 Vidéos réelles	10
5.11 Vidéos générées par intelligence artificielle	10
5.12 Vidéos artificiellement déformées	11
5.13 Vidéos de référence (Ground Truth)	12
6 Mise en œuvre de l’approche retenue	12
6.1 Architecture logicielle et exécution	12
6.2 Optimisation du temps de calcul	13
6.3 Gestion du cache et reproductibilité	14
7 Étude expérimentale	14
7.1 Expérience 1 : Choix de l’estimateur des paramètres intrinsèques	14
7.2 Expérience 2 : détection d’une image modifiée	16
7.3 Expérience 3 : Comparaison de paires réelles et générées (appariées)	17
7.4 Expérience 4 : Résultats sur l’ensemble du jeu de données	19
7.5 Expérience 5 : Classification supervisée	19
7.6 Expérience 6 : Apport des caractéristiques GeCo	20
7.7 Expérience 7 : Sensibilité à la fréquence d’images	20
7.8 Expérience 8 : Influence du prompt de la vidéo générée	21
8 Analyse et discussion	22
8.1 Synthèse des résultats	22
8.2 Limites de notre méthode	22
8.3 Perspectives d’amélioration	23

9 Conclusion	23
Références	25
A Annexes	26
A.1 Distributions des estimations sur la séquence hydrant	26
A.2 Résultats par paire	26

Table des figures

1	Positionnement de l'approche proposée par rapport aux principales familles de méthodes de détection.	4
2	Chaîne de reprojection géométrique entre deux images d'une même séquence. . .	7
3	Exemple d'occultation et d'incohérence lors de la réapparition d'un objet dans une vidéo générée.	9
4	Protocole de génération d'une séquence appariée à partir de la première et de la dernière image réelles.	11
5	Exemple de déformation locale par transformation Thin-Plate Spline sur la séquence toytruck.	12
6	Évolution de l'estimation de la distance focale sur la séquence hydrant avec VGGT et GeoCalib.	15
7	Détection temporelle d'une image artificiellement déformée dans la séquence toytruck.	17
8	Comparaison image par image pour la séquence bench : score réel 7,359 et score généré 12,058.	18
9	Comparaison image par image pour la séquence teddybear : score réel 3,776 et score généré 7,880.	18
10	Chaîne de classification supervisée à partir des caractéristiques extraites pour chaque image.	19
11	Erreur photométrique sur une même séquence échantillonnée à 24 et 12 images par seconde.	20
12	Évolution de l'erreur selon la complexité du prompt pour une même scène. . . .	21
13	Distribution des paramètres intrinsèques estimés par GeoCalib sur la séquence hydrant.	26
14	Distribution des paramètres intrinsèques estimés par VGGT sur la séquence hydrant.	26

Table des expériences

7.1	Expérience 1 : Choix de l'estimateur des paramètres intrinsèques	14
7.2	Expérience 2 : détection d'une image modifiée	16
7.3	Expérience 3 : Comparaison de paires réelles et générées (appariées)	17
7.4	Expérience 4 : Résultats sur l'ensemble du jeu de données	19
7.5	Expérience 5 : Classification supervisée	19
7.6	Expérience 6 : Apport des caractéristiques GeCo	20
7.7	Expérience 7 : Sensibilité à la fréquence d'images	20
7.8	Expérience 8 : Influence du prompt de la vidéo générée	21

1 Remerciements

Je tiens tout d’abord à remercier Emanuel Trabes pour ses conseils, sa disponibilité et la confiance qu’il m’a accordée. La liberté d’exploration dont j’ai bénéficié m’a permis de mener ce travail comme une véritable démarche de recherche, en formulant des hypothèses, en testant plusieurs pistes et en apprenant autant des résultats positifs que des limites rencontrées.

Je remercie également Carlos Valderrama et Jérôme Verscheuren pour m’avoir placé dans de très bonnes conditions de travail et pour avoir accompagné le déroulement du stage. Leur soutien a contribué à rendre cette expérience aussi formatrice sur le plan scientifique que sur le plan humain.

J’adresse mes remerciements à Leslie Gu (Harvard) et Junhwa Hur (Google DeepMind) pour les échanges autour de leurs travaux sur la cohérence géométrique des vidéos. Ces échanges ont nourri ma réflexion.

Enfin, je remercie mes camarades pour les discussions régulières autour du projet. Leurs questions et leurs points de vue m’ont aidé à prendre du recul, à clarifier certains choix et à envisager des solutions que je n’aurais pas nécessairement identifiées seul.

2 Note complémentaire

Ce travail a été conçu dans une optique de **reproductibilité**. L’ensemble des instructions d’installation et d’exécution est disponible dans le fichier README du dépôt GitHub : <https://github.com/FelixESIEEE2024/DeepFake-Detection-3Dvision>. Conformément aux recommandations de Carlos Valderrama, la publication du code source et du jeu de données est prévue à partir du **7 août**, afin de concentrer les efforts sur la finalisation et la précision du présent rapport dans un premier temps. Les commandes permettant de reproduire toutes les expériences présentées ci-dessous, ainsi que les fichiers de sortie correspondants (CSV), seront quant à elles mises à disposition sur un Drive partagé **au plus tard le 1er août**.

3 Introduction

Les progrès récents des modèles génératifs, tels que **StyleGAN** et les **modèles de diffusion** [1, 2], permettent aujourd’hui de produire des images et des vidéos d’un réalisme remarquable. Cette évolution rend la détection des deepfakes de plus en plus difficile et soulève d’importants enjeux sociétaux, notamment en matière de désinformation, d’usurpation d’identité et de manipulation de l’opinion publique. Les nombreuses vidéos falsifiées du président ukrainien **Volodymyr Zelensky** en constituent une illustration marquante.

Face à cette évolution, les approches traditionnelles fondées sur l’analyse des artefacts numériques montrent progressivement leurs limites. Ce projet explore donc une approche différente, basée sur le respect des lois physiques qui régissent une scène tridimensionnelle. Nous nous intéressons d’abord à la **cohérence photométrique**, puis à la **cohérence géométrique** entre plusieurs images d’une même scène.

3.1 Définition — Géométrie projective (Projective Geometry)

La géométrie projective est le cadre mathématique qui décrit la projection d’une scène 3D sur une image 2D. Elle étudie notamment les transformations de perspective, les points de fuite, les droites, les plans ainsi que les relations géométriques entre plusieurs vues d’une même scène.

3.2 Problématique

Les vidéos générées par intelligence artificielle présentent-elles des incohérences géométriques exploitables pour distinguer un contenu généré d’une vidéo authentique ?

3.3 Objectifs du projet

L’objectif de ce projet de recherche est de développer un pipeline capable d’évaluer la cohérence géométrique d’une vidéo à partir de techniques de vision 3D. Il vise à déterminer si les incohérences géométriques peuvent être exploitées pour détecter des vidéos générées par intelligence artificielle, tout en évaluant expérimentalement les performances, les limites et la robustesse de cette approche.

4 État de l’art et contexte scientifique

La détection de contenus générés par intelligence artificielle est devenue un domaine de recherche majeur avec les progrès récents des modèles génératifs de type *diffusion* et *transformers*. De nombreuses méthodes ont été proposées ces dernières années, chacune exploitant un type d’information différent afin de distinguer les contenus réels des contenus synthétiques.

Les premières approches reposaient principalement sur l’analyse des **artefacts numériques** laissés par les modèles génératifs. Elles exploitaient notamment des analyses fréquentielles (*Frequency Analysis*) [3], des méthodes d’*Error Level Analysis (ELA)* [4] ou encore des incohérences liées à la compression, au bruit du capteur ou aux statistiques des pixels. Ces méthodes étaient particulièrement efficaces sur les premières générations de deepfakes, qui introduisaient de nombreux défauts visibles dans les hautes fréquences.

Cependant, les modèles génératifs récents produisent des images et des vidéos de qualité beaucoup plus élevée. Une grande partie de ces artefacts a progressivement disparu, rendant les approches fondées uniquement sur l’analyse du contenu numérique de plus en plus difficiles à généraliser. Ce constat met en évidence un **research gap** : plutôt que d’analyser uniquement les pixels, il devient pertinent d’étudier si les contenus générés respectent réellement les lois physiques et géométriques qui régissent une scène tridimensionnelle.

À notre connaissance, très peu de travaux abordent la détection de deepfakes sous l’angle de la cohérence géométrique. Les recherches exploitant explicitement la géométrie 3D restent rares et se concentrent principalement sur des problématiques d’estimation de profondeur, de reconstruction 3D ou de pose caméra, sans traiter directement la discrimination entre vidéos réelles et générées. Pourtant, ces sujets constituent aujourd’hui des axes majeurs de la vision par ordinateur et sont au cœur d’applications industrielles **telles que la reconstruction tridimensionnelle pour le contrôle non destructif ou l’industrie aéronautique** [5]. Notre revue de littérature a ainsi davantage servi à comprendre ces techniques de pointe qu’à identifier des méthodes existantes de détection de deepfakes.

Au cours du projet, nous avons finalement identifié **GeCo (Geometry Consistency)**, développé par des chercheurs de **Google DeepMind** [6], comme le travail le plus proche de notre approche. GeCo ne cherche pas à détecter les deepfakes, mais à améliorer la qualité des vidéos générées en proposant une métrique différentiable de cohérence géométrique. Le principe consiste à reconstruire une scène en trois dimensions, à reprojeter les images entre elles puis à mesurer leur cohérence photométrique. Bien que l’objectif diffère du nôtre, plusieurs concepts développés dans cet article ont fortement inspiré notre pipeline. Ces contributions seront présentées plus en détail dans la suite de ce rapport.

5 Démarche scientifique et méthodologie

Avant de développer les premiers algorithmes, une part importante du projet a été consacrée à la compréhension des fondements de la vision par ordinateur. Ce travail s’est rapidement transformé en une véritable démarche exploratoire, où l’objectif n’était pas uniquement d’implémenter une méthode existante, mais de comprendre les principes physiques et géométriques qui permettent à un ordinateur d’interpréter une scène. Plusieurs semaines ont ainsi été consacrées à l’étude des notions de géométrie 3D, qui constituent le cœur de l’approche développée par la suite.

Cette phase d’apprentissage a également conduit à considérer une image non plus comme une photographie, mais comme un ensemble de pixels dont les valeurs sont directement liées aux propriétés physiques de la scène observée. Pour acquérir ces bases, je me suis principalement appuyé sur les huit cours de géométrie de la vision de Pascal Monasse, professeur du Master MVA de l’ENS Paris-Saclay [7], qui ont constitué le socle théorique de l’ensemble du projet.

5.1 Première approche : la cohérence photométrique

Avant d’exploiter les informations de géométrie 3D et de mouvement, nous avons d’abord cherché à détecter les deepfakes à partir d’une propriété physique plus fondamentale : la propagation de la lumière.

Cette première exploration est importante car elle explique le cheminement qui a conduit au pipeline final. Notre hypothèse était simple : une image n’est qu’un ensemble de pixels dont l’intensité dépend de la manière dont la lumière interagit avec les objets de la scène. **Si cette interaction obéit à des lois physiques précises, alors une image synthétique pourrait présenter des incohérences d’éclairage révélatrices d’un deepfake.**

Pour modéliser cette interaction, nous nous sommes appuyés sur le modèle de réflexion de Lambert. Une surface dite *lambertienne* diffuse la lumière de manière identique dans toutes les directions. Dans sa forme la plus simple, l’intensité observée au pixel p est donnée par :

$$I(p) = \rho(p) \max \left(0, n(p)^\top \ell \right) \quad (1)$$

- $\rho(p)$ représente l’albédo de la surface, c’est-à-dire sa capacité à réfléchir la lumière ;
- $n(p)$ est la normale unitaire à la surface ;
- ℓ correspond à la direction et à l’intensité de la source lumineuse ;
- l’opérateur $\max(0, \cdot)$ garantit qu’une surface orientée à l’opposé de la lumière ne reçoit aucune illumination.

Cette relation constitue le fondement des méthodes de *Shape from Shading*, dont l’objectif est de reconstruire la géométrie d’une scène à partir des variations d’intensité observées dans une image [8]. Elle offrait ainsi un premier moyen d’associer une loi physique aux valeurs des pixels.

L’idée était alors d’utiliser cette loi comme outil de détection. À partir d’une image, le pipeline estimait les différents paramètres du modèle (géométrie, albédo et éclairage), puis reconstruisait une image théorique en recalculant l’intensité de chaque pixel à l’aide de l’équation précédente. Cette image reconstruite était ensuite comparée à l’image d’origine.

La comparaison reposait sur l’erreur photométrique moyenne absolue (*Mean Absolute Photometric Error*, MAE). Cette métrique quantifie, pixel par pixel, la différence d’intensité entre l’image observée et l’image reconstruite ; elle est calculée comme suit :

$$\text{MAE} = \frac{1}{N} \sum_{p=1}^N |I_{\text{observée}}(p) - I_{\text{reconstruite}}(p)| \quad (2)$$

où N désigne le nombre total de pixels. L'hypothèse était qu'une image réelle respecterait davantage le modèle physique et présenterait donc une erreur faible, tandis qu'une image générée artificiellement produirait une erreur plus importante.

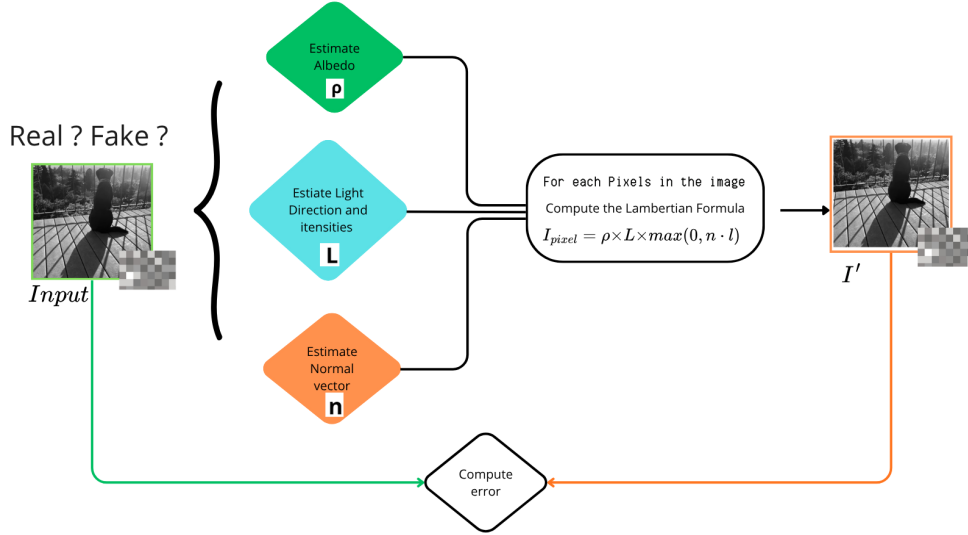


Figure 1 – Positionnement de l’approche proposée par rapport aux principales familles de méthodes de détection.

5.2 Discussion de la première méthode

Cette première exploration a toutefois rapidement révélé les limites d’une telle approche. Une même zone sombre peut provenir d’une surface orientée à l’opposé de la lumière, d’un matériau peu réfléchissant, d’une ombre portée ou d’une source lumineuse peu intense. À partir d’une seule image, il est très difficile de distinguer ces différentes causes. De plus, le modèle lambertien repose sur plusieurs hypothèses simplificatrices, comme des surfaces parfaitement mates et un éclairage unique, rarement vérifiées dans des scènes réelles où les reflets spéculaires, les transparences et les sources lumineuses multiples sont fréquents.

Cette étude a également permis de mieux comprendre la nature du problème. La reconstruction reposait entièrement sur des estimations : géométrie, albédo et éclairage devaient être inférés à partir d’une unique photographie. Or, prédire correctement les ombres nécessite de connaître la géométrie complète de la scène, la position des objets, des sources lumineuses et des surfaces recevant ces ombres. Une seule image ne contient donc pas suffisamment d’informations, si bien qu’une erreur photométrique élevée reflétait souvent les limites des estimations plutôt qu’une véritable incohérence liée à un deepfake.

Cette première exploration a néanmoins été déterminante pour la suite du projet. Elle a d’abord fait émerger une idée qui restera au cœur de notre approche : comparer des valeurs de pixels observées à des valeurs prédites par un modèle physique. Cette comparaison, matérialisée par l’erreur photométrique (*Photometric Error*), deviendra la principale métrique utilisée dans le reste du projet. Elle a également montré un point fondamental : une image seule contient trop

peu d’informations pour raisonner de manière fiable sur la cohérence physique d’une scène. **Ce constat a naturellement conduit à notre seconde approche, qui consiste à exploiter la vidéo**, qui fournit plusieurs observations d’une même scène et permet d’étudier non plus la cohérence photométrique d’une image, mais la cohérence géométrique entre plusieurs images successives.

5.3 Deuxième approche : la cohérence géométrique

Contrairement aux ombres, qui dépendent fortement des conditions d’éclairage, la structure tridimensionnelle d’un objet reste identique lorsqu’il est observé sous différents points de vue. Cette propriété est décrite par la **géométrie épipolaire**, l’un des fondements de la vision par ordinateur multi-vue.

Lorsqu’une scène est observée depuis deux positions différentes, la position d’un point dans la première image ne peut pas être arbitraire dans la seconde. Une fois la position du point connue dans la première image, sa position possible dans la seconde est contrainte à appartenir à une **ligne épipolaire** [7]. Cette relation constitue le principe central de la géométrie multi-vue.

En coordonnées homogènes, cette contrainte s’écrit :

$$\tilde{x}_2^\top F \tilde{x}_1 = 0 \quad (3)$$

où F désigne la **matrice fondamentale**, qui encode la géométrie entre les deux vues.

Notre stratégie pour vérifier si cette loi est respectée dans nos vidéos consiste à **reconstruire la position 3D de chaque pixel d’une image afin de prédire où il devrait apparaître dans l’image suivante**. Si la géométrie de la scène est correctement estimée, cette image reconstruite doit être très proche de l’image réellement observée. À l’inverse, des incohérences géométriques entraînent des différences visibles lors de cette reprojection. Pour atteindre cet objectif, nous avons retenu une approche dense, proche des méthodes d’odométrie visuelle [22]. Contrairement aux approches classiques qui ne suivent que quelques points d’intérêt, notre méthode exploite directement l’intensité d’un grand nombre de pixels afin d’estimer le mouvement de la caméra et de reconstruire la scène avec la plus grande précision possible. La métrique de l’approche précédente est conservée.

5.4 Pour réaliser cette reprojection, trois informations sont nécessaires :

- une **carte de profondeur**, qui permet de positionner chaque pixel dans l’espace tridimensionnel ;
- les **paramètres intrinsèques** de la caméra, qui décrivent sa géométrie interne ;
- les **paramètres extrinsèques**, c’est-à-dire la rotation et la translation entre les deux vues.

L’estimation de ces trois quantités constitue le cœur scientifique de notre pipeline (et de notre projet). Il s’agit également d’un problème majeur de la recherche actuelle en vision 3D, puisque les caméras ne capturent que des images bidimensionnelles alors que les applications, comme les véhicules autonomes, la robotique ou la réalité augmentée, doivent raisonner dans un monde tridimensionnel.

5.5 Estimation de la profondeur

La profondeur constitue le premier élément indispensable de notre pipeline. En effet, pour reprojeter correctement un pixel d'une image vers une autre, il ne suffit pas de connaître sa position dans le plan image : il est également nécessaire de connaître sa position dans l'espace tridimensionnel. Sans cette information, il est impossible de reconstruire la géométrie de la scène et de prédire où chaque pixel devrait apparaître dans la vue suivante.

Pour estimer cette profondeur, nous utilisons **Depth Anything** [9], l'un des modèles de référence actuels pour l'estimation monoculaire de profondeur. Celui-ci est construit à partir de **DINOv2** [10], un modèle de vision auto-supervisé (*self-supervised*) entraîné sur plusieurs centaines de millions d'images sans annotations. Contrairement à un réseau de classification classique, DINOv2 n'apprend pas à reconnaître des catégories d'objets, mais à produire une représentation riche de chaque image en capturant sa structure, sa géométrie et son contenu sémantique. Des régions appartenant au même objet ou partageant des caractéristiques similaires obtiennent ainsi des représentations proches dans l'espace latent.

Depth Anything exploite ces représentations comme point de départ et entraîne une tête de prédiction dédiée à la profondeur. Grâce aux informations géométriques et sémantiques déjà apprises par DINOv2, le modèle est capable de produire une **carte dense de profondeur**, c'est-à-dire une estimation de la distance pour chacun des pixels de l'image. Cette carte constitue ensuite l'entrée principale de notre pipeline de reprojection géométrique.

Une fois ces trois quantités estimées, nous pouvons appliquer notre algorithme. Dans le modèle de caméra sténopé (*Pinhole Camera Model*), les paramètres intrinsèques sont regroupés dans la matrice :

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (4)$$

où f_x et f_y représentent les distances focales exprimées en pixels, et (c_x, c_y) le point principal de l'image.

À partir d'un pixel homogène $\tilde{p} = (u, v, 1)^\top$ et de sa profondeur $Z(p)$, il est alors possible de reconstruire sa position dans le repère de la caméra source :

$$X_s(p) = Z(p) K_s^{-1} \tilde{p} \quad (5)$$

Le point est ensuite transformé dans le repère de la caméra cible à l'aide de la rotation R et de la translation t :

$$X_t(p) = R X_s(p) + t \quad (6)$$

Enfin, ce point 3D est reprojété dans la seconde image :

$$p' = \pi(K_t X_t(p)) \quad (7)$$

avec

$$\pi(x, y, z)^\top = \left(\frac{x}{z}, \frac{y}{z} \right)^\top. \quad (8)$$

Appliqué à une image, le pipeline suit donc le schéma présenté ci-dessous.

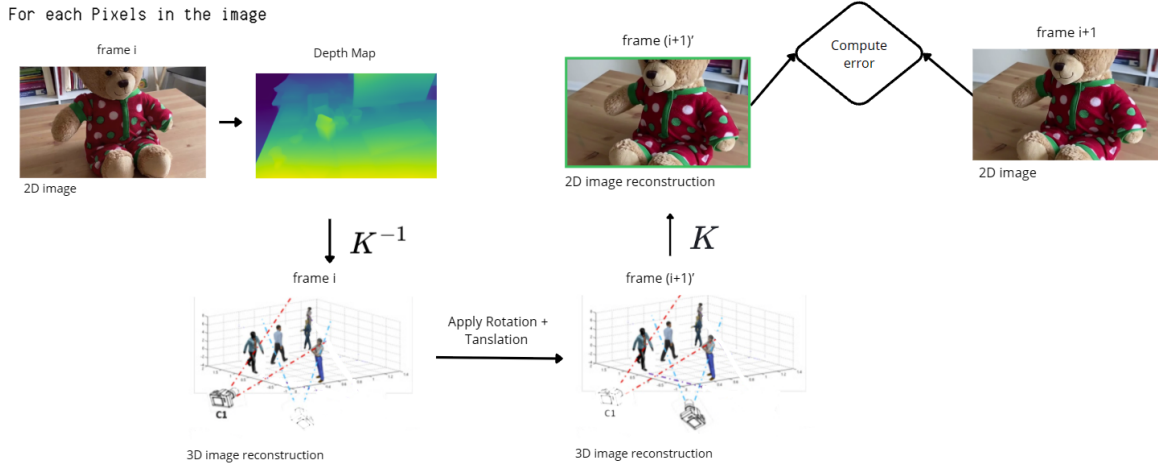


Figure 2 – Chaîne de reprojection géométrique entre deux images d’une même séquence.

5.5.1 Estimation des paramètres extrinsèques

Les paramètres extrinsèques décrivent le déplacement de la caméra entre deux images, c’est-à-dire sa rotation et sa translation. Leur estimation est essentielle : une erreur sur la pose décale l’ensemble des pixels reprojétés et augmente artificiellement l’erreur photométrique. Plutôt que d’utiliser un second réseau de neurones, nous avons choisi une **optimisation géométrique directe**, qui exploite uniquement les informations présentes dans les images et limite la dépendance aux modèles d’intelligence artificielle.

La pose est représentée dans le groupe de Lie $SE(3)$ [7], composé de six paramètres : trois pour la rotation et trois pour la translation. À chaque itération, le résidu photométrique est linéarisé autour de la pose courante :

$$r(\xi + \delta\xi) \approx r(\xi) + J \delta\xi \quad (9)$$

où J est la Jacobienne du résidu par rapport aux paramètres de pose. La mise à jour est ensuite calculée à l’aide de l’algorithme de **Gauss-Newton** :

$$H = J^\top J, \quad g = J^\top r, \quad (H + \lambda I) \delta\xi = -g \quad (10)$$

Le terme λI stabilise la résolution lorsque les informations géométriques sont insuffisantes. La nouvelle pose est acceptée si elle réduit l’erreur photométrique, puis appliquée à l’aide de la carte exponentielle de $SE(3)$.

Afin d’améliorer la robustesse, l’optimisation est réalisée selon une **pyramide multi-échelle**. Les images sont successivement traitées à plusieurs résolutions : les niveaux les plus grossiers permettent d’estimer les grands déplacements, tandis que les niveaux les plus fins affinent progressivement l’alignement. À chaque niveau, le pipeline reconstruit les points 3D, les transforme selon la pose courante, les reprojette dans l’image cible, calcule l’erreur photométrique, puis met à jour la pose.

Cette approche suppose toutefois une luminosité relativement constante entre les images ainsi qu’une scène majoritairement rigide. Ses performances diminuent en présence de changements d’éclairage importants, de mouvements indépendants ou de déplacements trop importants entre deux images.

5.5.2 Estimation des paramètres intrinsèques

Les **paramètres intrinsèques** décrivent les caractéristiques internes de la caméra ayant capturé l'image, notamment sa distance focale et son point principal. Ils permettent de relier les coordonnées d'un pixel à un rayon dans l'espace 3D et sont donc indispensables pour reconstruire correctement la géométrie de la scène et réaliser la reprojection.

L'estimation de ces paramètres constitue aujourd'hui un défi majeur en vision par ordinateur. Peu de jeux de données publics fournissent les véritables paramètres de caméra (*ground truth*), et les vidéos capturées avec des smartphones ne les enregistrent généralement pas. Cette difficulté est encore plus marquée pour les vidéos générées par modèles de diffusion, qui ne proviennent d'aucune caméra réelle : leurs paramètres intrinsèques n'existent donc pas et doivent être estimés.

À la suite de plusieurs discussions avec **Leslie Gu** et **Junhwa Hur**, deux chercheurs spécialisés dans ce domaine, deux méthodes de l'état de l'art ont été retenues : **GeoCalib** et **VGGT**. Une étude comparative, présentée dans la partie expérimentale, a ensuite été menée afin d'identifier la méthode la plus adaptée à notre pipeline et de comprendre les raisons expliquant leurs différences de performances.

GeoCalib estime les paramètres intrinsèques à partir d'une seule image [11]. Le modèle repose sur **SegNeXt**, un réseau convolutif qui prédit des informations géométriques telles que la position de l'horizon et la direction verticale de chaque pixel. Ces prédictions sont ensuite utilisées par l'algorithme d'optimisation de **Levenberg-Marquardt**, qui recherche les paramètres de caméra minimisant l'écart entre la géométrie observée et celle prédite par le modèle :

$$\theta^* = \arg \min_{\theta} \sum_i \|g_i^{obs} - g_i(\theta)\|^2 \quad (11)$$

où θ représente les paramètres intrinsèques de la caméra. La distance focale influence principalement le champ de vision, tandis que la distorsion contrôle la courbure des lignes en périphérie de l'image.

VGGT adopte une approche différente [12]. Les images sont découpées en *patches*, transformées en *tokens* puis traitées par un **Vision Transformer**, capable d'exploiter simultanément plusieurs vues d'une même scène. Un *camera token* est associé à chaque image afin d'encoder les informations relatives à sa caméra. À partir de cette représentation, le réseau prédit directement les paramètres intrinsèques et extrinsèques en une seule passe. Les prédictions sont apprises en minimisant une **Huber Loss** entre les paramètres estimés et les paramètres de référence :

$$\mathcal{L} = \text{Huber}(\hat{\theta}, \theta) \quad (12)$$

Contrairement à GeoCalib, VGGT ne repose donc pas sur une étape d'optimisation géométrique explicite : les paramètres de caméra sont directement inférés par le réseau.

5.6 Amélioration de la robustesse

Malgré les résultats encourageants obtenus avec notre pipeline de reprojection photométrique, plusieurs limites sont rapidement apparues. Certaines erreurs provenaient non pas d'incohérences géométriques, mais de phénomènes naturels tels que les occultations ou les sorties du champ de la caméra. Il devenait donc nécessaire de rendre la métrique plus robuste avant d'évaluer ses performances sur des vidéos générées.

Dans cette optique, nous nous sommes appuyés sur **GeCo** [6], présenté précédemment dans l'état de l'art. Contrairement à notre travail, GeCo n'a pas été conçu pour détecter les deepfakes. Son

objectif est de construire une **métrique différentiable** de cohérence géométrique pouvant être utilisée comme fonction de coût lors de l'entraînement de modèles de génération vidéo. L'idée retenue dans ce stage a donc été de **détourner cette métrique** afin de l'utiliser comme outil d'analyse de vidéos réelles et générées.

Le principe général reste proche du nôtre : reconstruire une scène en trois dimensions, la reprojeter puis mesurer les incohérences observées. En revanche, GeCo ne compare pas directement les intensités des pixels. Le modèle s'appuie sur le **flot optique** (*Optical Flow*), c'est-à-dire le déplacement apparent de chaque pixel entre deux images successives. Par exemple, lorsqu'une caméra se déplace vers la droite, un arbre observé dans la scène verra sa position se déplacer progressivement vers la gauche dans l'image. Le flot optique associe ainsi à chaque pixel un vecteur décrivant ce déplacement. GeCo reprojette ensuite ces vecteurs afin de vérifier si le mouvement prédit est cohérent avec le mouvement réellement observé. Comme dans notre pipeline, l'idée consiste finalement à comparer une valeur prédite à une valeur observée ; seule la grandeur physique étudiée est différente.

L'article introduit également plusieurs améliorations destinées à renforcer la robustesse de cette métrique. Les paramètres intrinsèques, la profondeur et la pose de la caméra sont estimés à l'aide de **VGGT**, tandis que l'évaluation n'est plus réalisée uniquement entre deux images successives mais sur une **fenêtre temporelle** de plusieurs images, ce qui permet de lisser les erreurs locales et d'obtenir une mesure plus stable.

L'une des principales contributions de GeCo concerne toutefois la **gestion des occultations**, qui constitue l'un des principaux défis de la génération vidéo actuelle.

5.7 Gestion de la validité des pixels

Avant de calculer l'erreur de reprojection, il est nécessaire d'identifier les pixels pour lesquels la comparaison est réellement pertinente. Deux situations rendent un pixel invalide.

La première correspond aux **sorties du champ de vision**. Lors du déplacement de la caméra, certains pixels sont reprojétés en dehors de l'image suivante. Ne possédant plus de correspondance observable, ils sont exclus du calcul de l'erreur photométrique afin d'éviter de pénaliser artificiellement le pipeline.

La seconde situation concerne les **occultations**. Une occultation apparaît lorsqu'un objet auparavant visible est temporairement masqué par un autre avant de réapparaître quelques images plus tard. Sans traitement spécifique, le pipeline comparerait alors des pixels représentant des objets différents, ce qui générerait une erreur élevée alors que la géométrie de la scène reste parfaitement cohérente.



Figure 3 – Exemple d'occultation et d'incohérence lors de la réapparition d'un objet dans une vidéo générée.

Ce phénomène constitue aujourd'hui l'une des principales difficultés des modèles de génération vidéo. Les incohérences apparaissent souvent lorsqu'un objet réapparaît après avoir été masqué. Elles restent généralement discrètes et passent facilement inaperçues à l'œil humain, car elles ne

provoquent pas de rupture visible dans la vidéo. La figure correspondante illustre ce phénomène sur une vidéo générée par **Sora** [13] : deux objets sont initialement visibles, puis l'un d'eux est masqué par une occultation avant que trois objets ne réapparaissent, révélant une incohérence géométrique.

Pour traiter ce problème, GeCo construit un **masque de validité** identifiant les pixels pouvant être comparés de manière fiable. Ce masque est obtenu en reprojetant également la carte de profondeur afin de déterminer quels pixels deviennent invisibles puis réapparaissent au cours du temps. Les régions correspondant à des occultations sont alors exclues du calcul de la métrique, ce qui améliore significativement sa robustesse.

5.8 Intégration dans notre pipeline

Nous avons donc récupéré l'implémentation de GeCo afin d'en détourner l'usage dans notre contexte. Deux objectifs ont été poursuivis. Le premier consistait à évaluer si la métrique proposée par GeCo permettait, à elle seule, de distinguer des vidéos réelles de vidéos générées. Le second était de **fusionner cette métrique avec notre erreur photométrique**, afin d'enrichir le signal géométrique extrait par notre pipeline.

Notre hypothèse était que ces deux approches, reposant sur des informations complémentaires (intensité des pixels pour notre méthode et cohérence du flot optique pour GeCo) permettraient d'améliorer les performances de détection. Les résultats de ces différentes expérimentations sont présentés dans la section suivante.

5.9 Création du jeu de données

La constitution du jeu de données a représenté une part importante du travail de ce stage. Contrairement à un problème de classification classique, il n'existait aucun jeu de données directement adapté à notre protocole expérimental. La création du jeu de données est donc devenue un véritable défi scientifique, réalisé progressivement au fil de l'avancement du projet.

Notre objectif n'était pas uniquement d'obtenir un score de détection, mais de comprendre dans quelles situations notre pipeline fonctionnait ou échouait. Pour cela, le jeu de données a été conçu de manière à comparer des vidéos représentant des scènes aussi similaires que possible, tout en couvrant différents types d'environnements et de mouvements. Cette stratégie permettra, dans les expériences présentées par la suite, d'évaluer l'influence de nombreux facteurs, tels que la vitesse de déplacement de la caméra, la complexité de la scène ou encore la qualité de la génération. Les vidéos ont également été réparties en plusieurs catégories afin de mener des analyses spécifiques pour chacune d'elles.

5.10 Vidéos réelles

Les vidéos réelles proviennent du jeu de données **Co3D**, développé par **Meta AI** [14]. Ce jeu de données regroupe plusieurs milliers de vidéos d'objets capturés sous différents points de vue, avec des mouvements de caméra lents et des scènes majoritairement rigides. Ces caractéristiques en font un support particulièrement adapté à notre pipeline de reprojection géométrique.

5.11 Vidéos générées par intelligence artificielle

Comparer directement une vidéo réelle à une vidéo générée représentant une scène différente n'aurait que peu d'intérêt, les écarts observés pouvant simplement provenir du contenu de la scène. Afin de rendre cette comparaison pertinente, nous avons adopté le protocole suivant : pour chaque vidéo réelle issue de Co3D, la première et la dernière image sont extraites puis fournies au

modèle génératif, avec pour consigne de reconstruire la vidéo intermédiaire en reproduisant le même mouvement de caméra dans le même intervalle de temps.

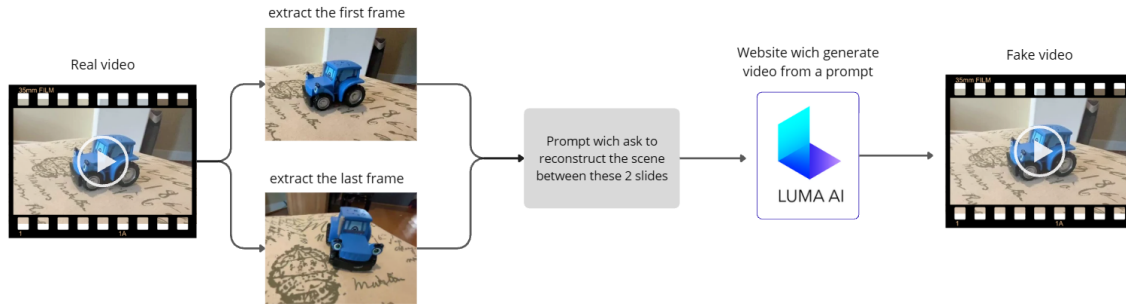


Figure 4 – Protocole de génération d’une séquence appariée à partir de la première et de la dernière image réelles.

Ce protocole ouvre également la voie à plusieurs expériences complémentaires. En faisant varier uniquement le **prompt**, il devient possible d’étudier son influence sur la cohérence géométrique de la vidéo produite. Certaines vidéos ont ainsi été générées plusieurs fois avec différentes consignes, par exemple en demandant explicitement au modèle de *respecter la géométrie de la scène*, afin d’observer l’impact de cette instruction sur notre métrique.

Les vidéos ont été générées avec **Dream Machine** de **Luma AI** [15], basé sur la famille propriétaire de modèles **Ray**. Bien que son architecture et son nombre de paramètres ne soient pas publics, il s’agit d’un modèle fermé considéré comme l’un des principaux modèles de génération vidéo actuels. Son utilisation a également été motivée par la disponibilité de crédits gratuits permettant de produire un grand nombre de vidéos.

5.12 Vidéos artificiellement déformées

Enfin, une dernière catégorie a été créée afin de disposer d’un protocole de validation parfaitement contrôlé. À partir des vidéos réelles de Co3D, une unique image est artificiellement déformée à l’aide d’une transformation **Thin-Plate Spline (TPS)** [16], suivant une approche inspirée de GeCo [6]. Cette transformation déplace plusieurs points de contrôle de l’image puis interpole une déformation lisse sur la région concernée, créant ainsi une incohérence géométrique localisée tout en conservant un aspect visuellement réaliste.

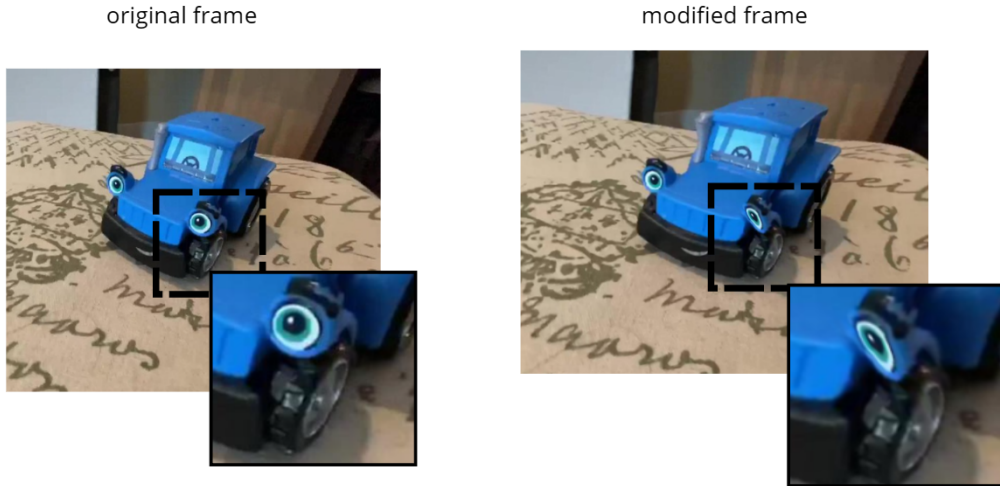


Figure 5 – Exemple de déformation locale par transformation Thin-Plate Spline sur la séquence toytruck.

L’objectif est ensuite de vérifier que notre pipeline attribue le score d’erreur le plus élevé à cette image modifiée. La image détectée est définie par

$$\hat{t} = \arg \max_t M(t), \quad (13)$$

où $M(t)$ désigne la métrique géométrique calculée à l’image t . Si l’image prédite $\hat{t}$ correspond à celle qui a été artificiellement déformée, la détection est considérée comme correcte.

5.13 Vidéos de référence (Ground Truth)

Quelques vidéos synthétiques de référence (*Ground Truth*) ont également été générées avec **Blender**. Bien que leur nombre soit limité, elles offrent un environnement parfaitement contrôlé et permettent de vérifier que les résultats obtenus par le pipeline restent cohérents lorsque la géométrie de la scène est entièrement connue.

6 Mise en œuvre de l’approche retenue

6.1 Architecture logicielle et exécution

Le pipeline a été développé en Python. Ce choix facilite l’intégration des modèles de profondeur et de calibration, la manipulation des tableaux avec NumPy, l’utilisation de PyTorch sur GPU et la production de résultats tabulaires.

Le dépôt a été organisé de manière modulaire afin de séparer la préparation des données, l’estimation géométrique, le calcul des erreurs et la production des résultats. Cette organisation permet d’exécuter indépendamment les différentes méthodes d’évaluation ou de les combiner au sein d’un même traitement.

- L’estimation de la **pose relative** entre deux images repose sur une optimisation de Gauss-Newton réalisée à plusieurs niveaux de résolution. Les transformations de caméra sont représentées dans le groupe de Lie ($SE(3)$), ce qui permet de manipuler conjointement les rotations et les translations.
- Les paramètres **intrinsèques** de référence ont d’abord été récupérés à partir des estimations

de VGGT disponibles dans le jeu de données Co3D. En complément, le calcul des intrinsèques a été automatisé avec GeoCalib pour chaque image, avant l'exécution du pipeline principal. Il est ainsi possible de choisir la méthode de calibration utilisée lors du lancement d'une expérience.

- Les cartes de profondeur sont obtenues avec **Depth Anything**, exécuté localement à partir de chaque image RGB. Le modèle produit une estimation de profondeur relative, ensuite redimensionnée à la résolution de l'image et convertie dans le format de profondeur inverse utilisé par l'odométrie visuelle. Toutes les cartes de profondeur d'une vidéo sont calculées une seule fois, puis conservées en mémoire pour être réutilisées.

Une fois les images, les profondeurs et les paramètres intrinsèques préparés, les images sont regroupées en fenêtres temporelles. La pose et l'erreur photométrique sont ensuite calculées pour les différentes paires de images sélectionnées dans chaque fenêtre.

Les occultations sont traitées à l'aide d'un *z-buffer*, qui conserve la profondeur du point visible le plus proche pour chaque pixel projeté. Les points masqués par une surface plus proche sont indiqués comme invalides dans le masque de visibilité et sont exclus du calcul de l'erreur.

L'interface en ligne de commande repose sur **argparse**. Elle permet notamment de sélectionner le dossier d'entrée, la catégorie de vidéos, la méthode d'intrinsèques, la taille de la fenêtre, l'écart entre deux images, la cadence cible, le nombre de niveaux de pyramide, les méthodes de score à activer et le dossier de sortie. Une expérience est ainsi décrite par sa configuration plutôt que par une modification du code. Cette organisation a été indispensable pour conduire les ablations sans créer plusieurs versions divergentes du programme.

Chaque exécution produit quatre niveaux de **fichiers CSV** : résultats par pixel ou région lorsque cela est nécessaire, agrégation par image, agrégation par fenêtre et agrégation par vidéo. Un fichier supplémentaire enregistre la configuration complète. Les résultats de notre métrique et ceux de GeCo sont rapprochés à partir du nom de la vidéo et de l'indice de l'image, puis fusionnés dans une table commune. Des courbes temporelles, des distributions et des cartes thermiques sont également générées.

6.2 Optimisation du temps de calcul

Le coût d'une méthode dense augmente rapidement : une image proche du mégapixel contient près d'un million de points à rétroprojeter, transformer, interpoler et accumuler. Dans la version Python initiale, le calcul de pose dominait donc très largement le temps d'exécution. Les boucles critiques ont été compilées à la volée avec Numba et le décorateur `@jit` [17]. Cette optimisation concerne la projection 3D-2D, l'interpolation bilinéaire, le calcul des résidus photométriques et l'accumulation du gradient et de la Hessienne. Les opérations sont exécutées en code machine natif sur le CPU, sans le surcoût de l'interpréteur Python à chaque pixel. La compilation utilise `fastmath=False` et conserve l'ordre de parcours et d'accumulation de la version de référence. Ce choix évite les réassociations flottantes susceptibles de modifier les résultats. Les erreurs, les Jacobiennes, les Hessiennes et les cartes photométriques produites par la version optimisée ont ainsi pu être comparées à celles de l'implémentation initiale. Les copies profondes des structures contenant les pyramides d'images et de profondeur ont également été remplacées par des copies légères : seule la pose varie pendant l'optimisation. Enfin, les transferts entre CPU et GPU ont été limités. Les images d'une fenêtre sont chargées successivement, leurs profondeurs sont calculées en un lot cohérent, puis conservées en mémoire pendant les comparaisons.

Le calcul de pose a été accéléré de 96,7 %, ce qui réduit le temps total de 64,7 %. Depth Anything

Étape	Avant optimisation	Après optimisation	Gain
Depth Anything complet	7,45 s/image	7,41 s/image	0,5 %
Calcul de pose	18,00 s/paire	0,59 s/paire	96,7 %
Algorithme complet	25,45 s/image	8,99 s/image	64,7 %

demeure presque inchangé, car cette étape était déjà exécutée dans un cadre optimisé. Les expériences finales ont été lancées sur une instance RunPod équipée d'un GPU NVIDIA RTX 5090, de 16 vCPU AMD Ryzen 9 9950X, de 92 Go de mémoire et d'un disque de conteneur de 30 Go.

6.3 Gestion du cache et reproductibilité

Le cache représente un compromis entre vitesse et stockage. Conserver toutes les cartes de profondeur accélérerait les exécutions futures, mais saturerait rapidement le disque ; tout recalculer augmenterait fortement le temps de traitement. Le programme ne conserve donc que les cartes nécessaires à l'itération courante. Elles sont réutilisées pour toutes les paires d'une fenêtre, puis supprimées lorsqu'elles ne sont plus utiles. Cette stratégie réduit également les échanges GPU–CPU : une fenêtre d'images est chargée, toutes ses profondeurs sont calculées, puis les calculs de pose sont exécutés. Les fichiers de configuration, les graines aléatoires lorsqu'elles interviennent et les agrégations CSV permettent de relier chaque résultat aux paramètres qui l'ont produit. Les instructions du dépôt complètent ce mécanisme pour permettre la reproduction des expériences.

7 Étude expérimentale

7.1 Expérience 1 : Choix de l'estimateur des paramètres intrinsèques

Les deux méthodes étudiées estiment les paramètres intrinsèques **indépendamment pour chaque image**. Or, dans notre jeu de données, les vidéos ne comportent **aucun zoom**. Les paramètres intrinsèques de la caméra sont donc supposés rester constants tout au long de la séquence. En théorie, un bon estimateur doit ainsi fournir des valeurs très proches d'une image à l'autre. Cette propriété nous permet d'évaluer directement la stabilité des deux méthodes.

Cette expérience est réalisée sur un échantillon de **100 vidéos réelles**. L'objectif est double : comparer la stabilité des estimateurs, puis mesurer leur impact sur les performances du pipeline de reprojection photométrique.

Évaluation de la stabilité des estimateurs.

Pour chaque vidéo, les différentes composantes de la matrice intrinsèque sont estimées sur l'ensemble des images. La moyenne et l'écart-type de chaque paramètre sont ensuite calculés, puis comparés aux quelques *ground truths* disponibles.

Les résultats montrent que **VGGT** fournit des estimations très stables, avec un écart-type moyen de seulement **9,44 pixels**. À l'inverse, **GeoCalib** présente un écart-type moyen de **705 pixels**, révélant une forte variabilité entre les images alors que les paramètres devraient théoriquement rester constants.

La séquence *hydrant* illustre particulièrement bien ce phénomène.

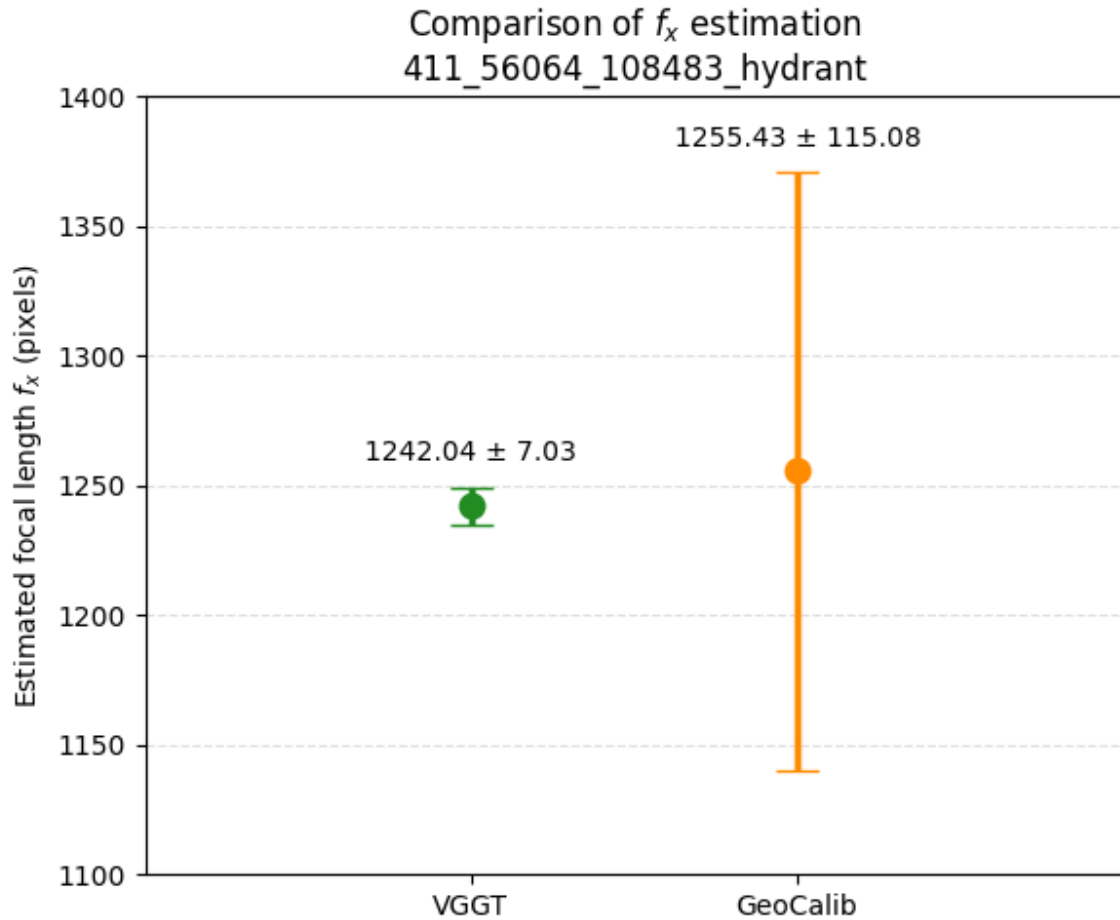


Figure 6 – Évolution de l'estimation de la distance focale sur la séquence hydrant avec VGGT et GeoCalib.

La figure ci-dessus compare les estimations de la distance focale f_x obtenues par VGGT et GeoCalib sur une même séquence. Les deux méthodes produisent des valeurs moyennes très proches (**1242,04 px** pour VGGT contre **1255,43 px** pour GeoCalib), mais GeoCalib présente une dispersion nettement plus importante (**écart-type de 115,08 px** contre **7,03 px** pour VGGT). Un graphique présentant la distribution complète des estimations est fourni en annexe afin d'illustrer ce même phénomène sous un autre angle.

Stratégie de moyennage et impact sur les performances.

Afin de réduire cette variabilité, nous calculons ensuite la moyenne des paramètres intrinsèques estimés sur l'ensemble de la vidéo. Cette opération améliore significativement les résultats de GeoCalib : les paramètres obtenus ne diffèrent plus que d'environ **120 pixels** de ceux estimés par VGGT.

Nous évaluons ensuite l'impact de chaque estimateur sur les performances du pipeline complet au moyen d'une **ablation study**. Cette expérience est réalisée uniquement sur des **vidéos réelles**, pour lesquelles aucune incohérence géométrique n'est attendue. Dans ce contexte, **plus l'erreur photométrique est faible, plus les paramètres intrinsèques estimés sont proches de la réalité**. Cette métrique permet donc de comparer objectivement la qualité des différents estimateurs. Quatre configurations sont ainsi évaluées :

Ces résultats montrent que le choix de la méthode d'estimation des paramètres intrinsèques

Méthode	Erreur absolue moyenne
VGGT	5,288
VGGT Mean	5,224
GeoCalib	9,297
GeoCalib Mean	5,397

influence directement les performances du pipeline. **VGGT** obtient les erreurs photométriques les plus faibles, tandis que les estimations brutes de **GeoCalib** dégradent fortement la qualité de la reprojection. En revanche, le simple moyennage des estimations de GeoCalib fait chuter son erreur de **9,297** à **5,397**, confirmant que sa principale limite réside dans son manque de stabilité temporelle.

Méthode retenue pour les expériences suivantes.

L'étude des deux articles permet d'expliquer cette différence. **VGGT** exploite simultanément plusieurs vues d'une même scène pour estimer les paramètres de la caméra, alors que **GeoCalib** réalise une estimation indépendante sur chaque image. Cette utilisation de l'information temporelle explique probablement la meilleure stabilité observée avec VGGT.

Nous retenons donc **VGGT** comme estimateur des paramètres intrinsèques pour la suite du projet. Cette conclusion reste toutefois spécifique à notre pipeline vidéo : **GeoCalib** demeure une méthode de référence pour la calibration à partir d'une image unique, notamment lorsqu'il est nécessaire d'estimer également les paramètres de distorsion.

7.2 Expérience 2 : détection d'une image modifiée

Avant d'appliquer notre métrique à la détection de vidéos générées par IA, nous avons souhaité vérifier qu'elle était capable de détecter des incohérences géométriques introduites artificiellement dans des vidéos réelles. Pour cela, nous avons modifié localement une unique image de plusieurs vidéos du jeu de données Co3D, puis évalué si la métrique était capable d'identifier la image altérée.

Les résultats montrent que, sur un sous-ensemble de **100 vidéos**, notre méthode détecte correctement la image modifiée dans **73 %** des cas, contre **71,59 %** pour la métrique proposée par GeCo. Il convient toutefois de relativiser cette comparaison : notre objectif n'était pas de surpasser les méthodes existantes, mais simplement de valider le bon fonctionnement de notre métrique avant de poursuivre les expérimentations. De plus, cette évaluation n'a été réalisée que sur un sous-ensemble de 100 vidéos, alors que le jeu de données complet en contient plus d'un millier.

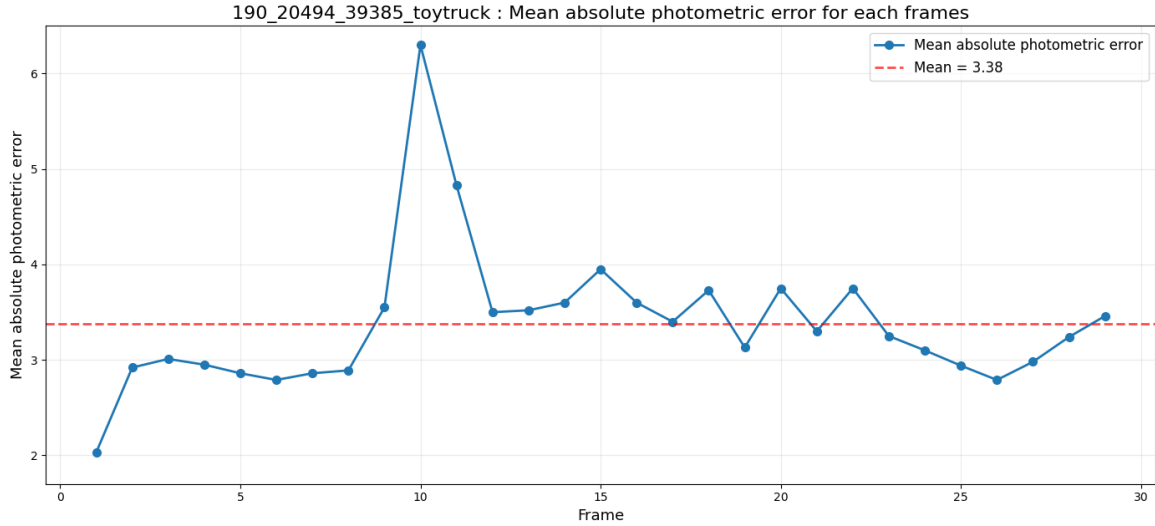


Figure 7 – Détection temporelle d’une image artificiellement déformée dans la séquence toytruck.

La figure correspondante présente un exemple obtenu sur la séquence **190_20494_39385_toytruck**. Malgré une modification relativement discrète et difficilement perceptible à l’œil nu, la métrique met clairement en évidence une augmentation locale de l’erreur de reprojection. Ce résultat est encourageant, car il montre que notre approche est sensible à des incohérences géométriques subtiles.

Cette expérience met également en évidence une propriété importante : les manipulations introduites par la TPS sont **locales**. Seule une petite région de l’image est modifiée, tandis que le reste de la scène demeure géométriquement cohérent. L’erreur se concentre donc autour de cette zone, mais son intensité est atténuée lorsqu’elle est moyennée sur l’ensemble de l’image.

Cette observation nous a conduits à envisager un calcul de la métrique sur plusieurs régions de l’image afin de détecter et localiser plus précisément les manipulations. Cette idée est particulièrement pertinente pour les deepfakes, où les modifications concernent le plus souvent une partie limitée de l’image, comme un visage remplacé (*face swap*) ou une expression faciale modifiée (*face reenactment*), tandis que le reste de la scène reste inchangé. En pratique, nous n’avons pas développé cette approche, car les **cartes d’erreur (heatmaps)** permettent déjà de visualiser efficacement la localisation des incohérences. Une étude du *Monde* estime par ailleurs que près de **80 %** des deepfakes concernent le visage, ce qui renforce l’intérêt d’une analyse localisée [19].

7.3 Expérience 3 : Comparaison de paires réelles et générées (appariées)

Nous avons ensuite évalué la méthode sur **33 paires** de vidéos, chacune étant composée d’une vidéo réelle et de sa version générée à partir de sa première et de sa dernière image. Les résultats obtenus sont encourageants. Pour **27 paires sur 33** (voir tableau en annexe), la vidéo générée présente une erreur photométrique plus élevée que la vidéo réelle. En moyenne, cette erreur augmente d’environ **30 %**, passant de **5,894** pour les vidéos réelles à **8,944** pour les vidéos générées. (la liste complète figure en annexe).

Exemples de séquences.

Pour la paire **415_57112_110099_bench**, l’erreur absolue moyenne est de 7,359 sur la vidéo réelle et de 12,058 sur la vidéo générée. La figure 6 montre que l’écart ne repose pas sur une seule image : la courbe générée reste généralement au-dessus de la courbe réelle et présente plusieurs pics importants.

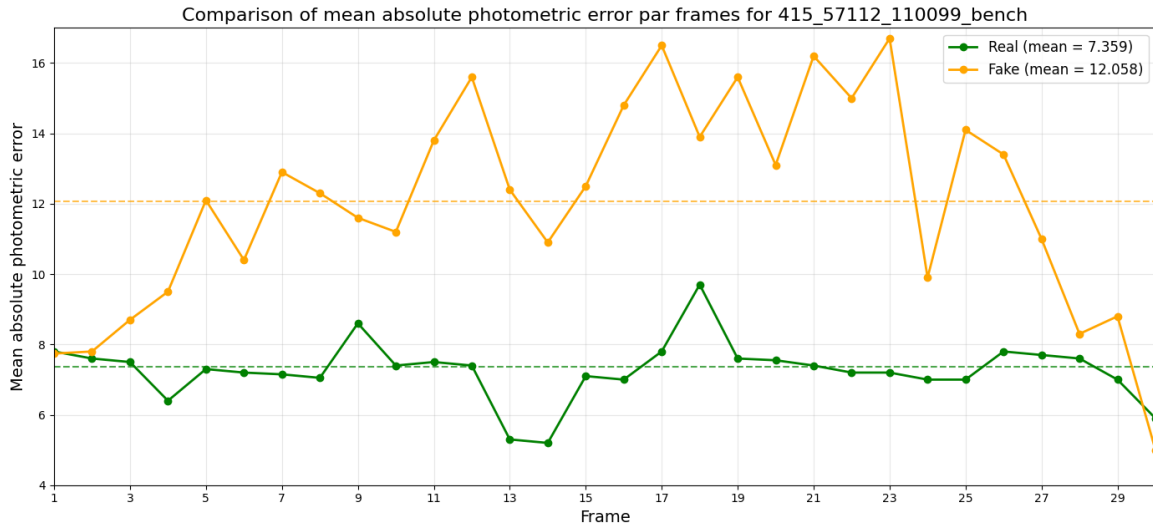


Figure 8 – Comparaison image par image pour la séquence bench : score réel 7,359 et score généré 12,058.

La paire 34_1479_4753_teddybear présente un écart encore plus marqué : 3,776 pour la vidéo réelle et 7,880 pour la vidéo générée. La courbe réelle reste basse, avec quelques variations ponctuelles, tandis que la courbe générée oscille autour d'un niveau supérieur.

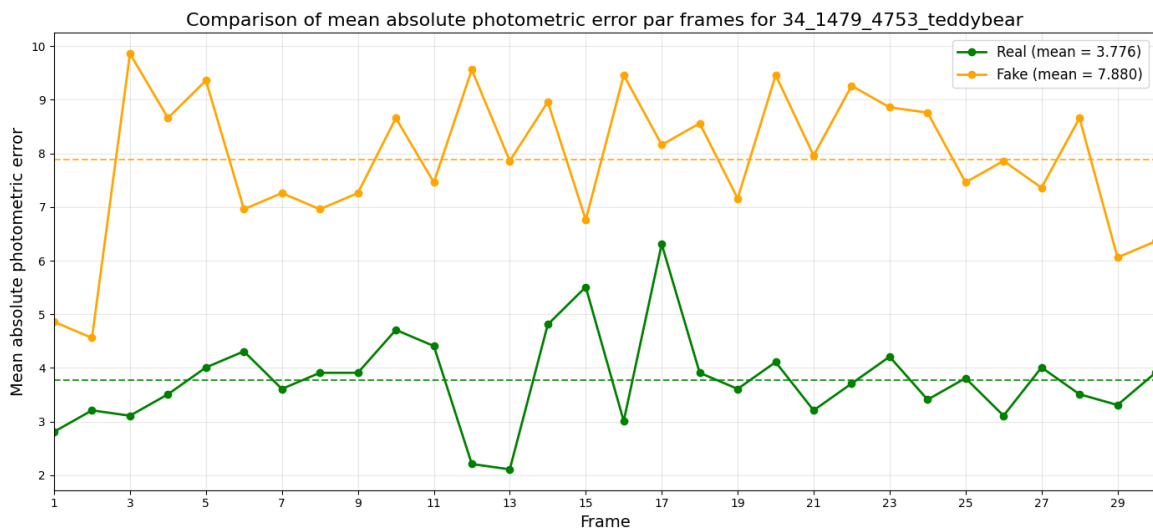


Figure 9 – Comparaison image par image pour la séquence teddybear : score réel 3,776 et score généré 7,880.

Cette expérience constitue un résultat central de ce travail, car elle apporte une première validation de l'hypothèse qui a guidé l'ensemble du projet. Les résultats montrent que les vidéos générées par IA présentent des incohérences géométriques mesurables, même lorsque celles-ci sont imperceptibles à l'œil humain. Autrement dit, bien que ces incohérences ne soient pas systématiquement visibles, elles laissent une signature physique détectable par une analyse quantitative. Ce constat justifie pleinement l'approche retenue et constitue le point de départ des développements présentés dans la suite de ce rapport.

7.4 Expérience 4 : Résultats sur l'ensemble du jeu de données

La comparaison globale, sans appariement, est beaucoup moins nette. L'erreur absolue moyenne atteint 5,9 pour les vidéos réelles, 6,16 pour les vidéos générées et 9,0 pour les vidéos artificiellement modifiées.

Groupe	MAE
Vidéos réelles	5.90
Vidéos générées par IA	6.16

Les vidéos modifiées forment un contrôle plus facile : la déformation est conçue pour créer un défaut ponctuel. En revanche, les moyennes réelle et générée sont proches et leurs dispersions se recouvrent. Un seuil fixe entre 5,90 et 6,16 produirait de nombreux faux positifs et faux négatifs. Cette apparente contradiction avec les paires est instructive. Dans une paire, une partie de la variabilité de scène est neutralisée. Dans l'ensemble global, une vidéo réelle difficile peut obtenir une erreur supérieure à une vidéo générée simple. Le score mesure alors autant la difficulté de la séquence (texture, vitesse, occultations et éclairage) que son origine. **La métrique est donc plus solide pour une comparaison relative que pour une décision absolue.**

7.5 Expérience 5 : Classification supervisée

On vient de voir qu'un tel seuil est fortement dépendant de la scène observée : certaines scènes produisent naturellement des erreurs de reprojection plus élevées que d'autres, ce qui rend l'utilisation d'un seuil fixe peu robuste.

Afin de s'affranchir de cette limitation, nous proposons d'aborder le problème comme une **tâche de classification supervisée**. Pour chaque image, l'ensemble des informations extraites par notre pipeline (erreur de reprojection, métriques issues de l'UFM, présence d'occlusions, etc.) est utilisé comme vecteur de caractéristiques. Un classifieur, tel qu'**ExtraTrees** [18], est ensuite entraîné à distinguer automatiquement les vidéos réelles des vidéos générées, sans avoir à définir manuellement un seuil de décision.

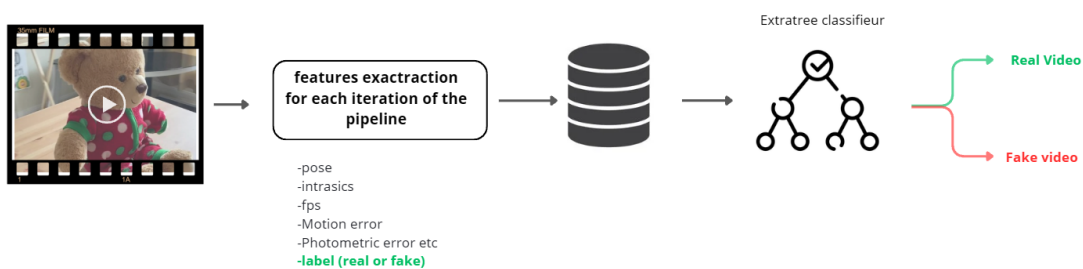


Figure 10 – Chaîne de classification supervisée à partir des caractéristiques extraites pour chaque image.

Chaîne de classification à partir des caractéristiques extraites pour chaque image.

Extra Trees construit un ensemble d'arbres de décision en randomisant fortement les variables et les seuils de séparation. Cette famille de modèles convient à un premier essai, car elle accepte des relations non linéaires, nécessite peu de prétraitement et fournit une importance approximative des caractéristiques.

La précision obtenue est d'environ 60 %. Ce résultat dépasse légèrement le hasard pour un problème binaire équilibré, mais il reste insuffisant pour valider une détection fiable. Les caractéristiques

actuelles n'expliquent pas assez bien la diversité des scènes. Elles sont également corrélées : plusieurs scores héritent des mêmes estimations de profondeur et de caméra, si bien que leur multiplication n'apporte pas nécessairement une information indépendante.

Cette expérience invalide donc, dans le protocole actuel, l'hypothèse selon laquelle un classifieur simple suffirait à apprendre automatiquement un seuil adapté à chaque scène. Elle ne montre pas que toute classification géométrique est impossible ; elle indique que davantage de données, une séparation rigoureuse par scène et des caractéristiques mieux normalisées sont nécessaires.

7.6 Expérience 6 : Apport des caractéristiques GeCo

L'intégration des sorties GeCo n'améliore pas de manière significative la classification globale. Le signal de mouvement résiduel et le signal de structure sont pertinents pour visualiser des défauts, mais ils reposent eux aussi sur des estimations de flot, de profondeur et de pose. Dans le jeu de données actuel, leur fusion avec l'erreur photométrique apporte donc une redondance plus qu'une séparation nette entre classes. L'apport principal de GeCo au projet est méthodologique. La distinction entre régions co-visibles et occultées a conduit à mieux définir les pixels valides. La reprojection de profondeur fournit une manière d'analyser la permanence d'un objet lorsqu'il disparaît puis réapparaît. Enfin, les jeux de données de déformations et d'occultations contrôlées proposés par les auteurs montrent l'importance d'évaluer séparément chaque type d'échec, plutôt que de résumer toutes les vidéos par un score global.

7.7 Expérience 7 : Sensibilité à la fréquence d'images

Pour la vidéo `*190_20494_39385_toytruck*` à durée d'observation identique (1,25 s), le passage de 24 FPS à 12 FPS réduit le nombre de images analysées de 30 à 15. Dans ces conditions, l'erreur moyenne passe de **3,24** à **4,38**, soit une augmentation de **35,2 %**.

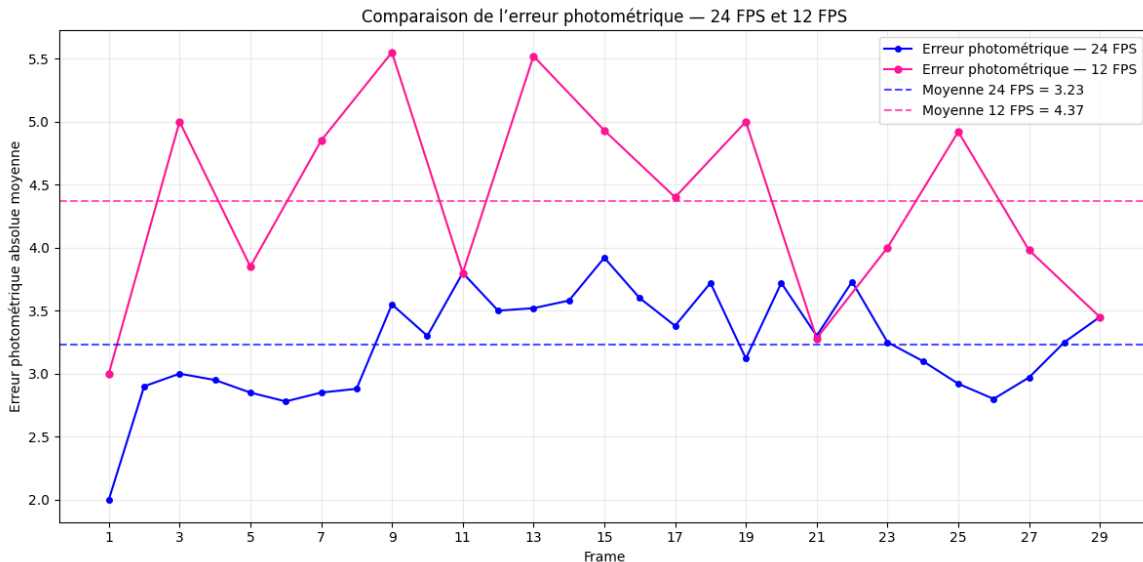


Figure 11 – Erreur photométrique sur une même séquence échantillonnée à 24 et 12 images par seconde.

Ce même phénomène a été observé sur un ensemble de **30 vidéos**, pour lesquelles la diminution de la fréquence d'acquisition entraîne une augmentation moyenne de l'erreur de **26 %**.

Ces résultats montrent que l'erreur dépend de la scène observée, mais également de la vitesse apparente du mouvement. En effet, diminuer le nombre d'images par seconde revient, du point de vue de l'algorithme, à augmenter l'écart temporel entre deux images successives, ce qui équivaut

à observer un mouvement de caméra plus rapide. Les estimations de profondeur, de pose et les opérations de reprojection deviennent alors plus difficiles, ce qui se traduit par une augmentation de l'erreur.

Conclusion. Ces résultats suggèrent que la vitesse apparente du mouvement constitue un facteur déterminant dans la qualité des estimations produites par notre pipeline. Plus le déplacement entre deux images est important, plus l'erreur de reprojection tend à augmenter. Par conséquent, toute comparaison quantitative entre différentes vidéos doit être réalisée sur des séquences présentant des vitesses de mouvement comparables, faute de quoi les écarts observés pourraient provenir de la dynamique de la scène plutôt que des performances intrinsèques de l'algorithme.

7.8 Expérience 8 : Influence du prompt de la vidéo générée

L'un des avantages de notre approche est qu'elle permet de comparer des vidéos représentant une scène très similaire. En modifiant uniquement le **prompt** utilisé pour générer une vidéo, il devient possible d'étudier l'influence de celui-ci sur la qualité géométrique du résultat produit par le modèle génératif, ici **Luma AI**.

La génération de vidéos étant particulièrement coûteuse en temps de calcul et en ressources matérielles, cette expérience a été menée sur un nombre limité d'exemples. Pour une même séquence (*190_20494_39385_1_toytruck*), cinq vidéos ont été générées à partir de prompts de plus en plus longs et précis. La figure correspondante présente l'évolution de l'erreur géométrique en fonction de la longueur du prompt.

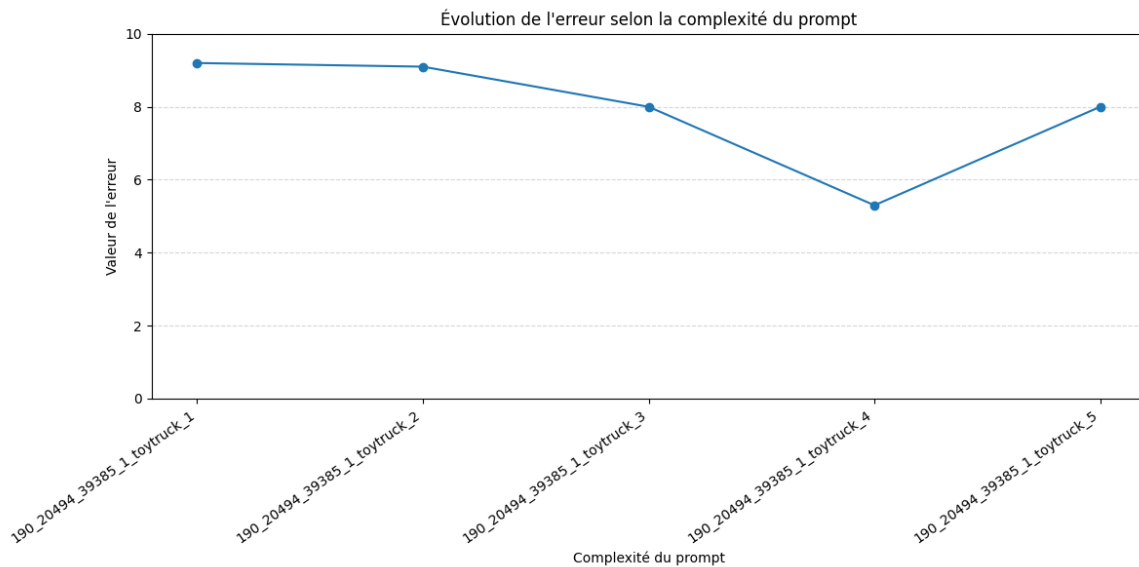


Figure 12 – Évolution de l'erreur selon la complexité du prompt pour une même scène.

Les résultats suggèrent qu'un **compromis** semble exister entre la liberté laissée au modèle et le niveau de précision des instructions. Un prompt très court laisse davantage de liberté au modèle, ce qui peut conduire à des incohérences géométriques. À l'inverse, un prompt trop détaillé semble parfois contraindre excessivement la génération et entraîner également une augmentation de l'erreur. Dans notre expérience, le prompt **190_20494_39385_1_toytruck_4** obtient les meilleurs résultats, avec un score se rapprochant de celui de la vidéo réelle.

Nous avons également observé que, dans **8 cas sur 10**, l'ajout de la consigne « *tu porteras une attention particulière à la cohérence géométrique entre les différentes images* » entraîne une diminution de l'erreur mesurée par notre métrique.

Remarque. Ces résultats montrent que **le prompt influence la qualité géométrique des vidéos générées**. Toutefois, le nombre d'essais reste toutefois trop faible pour conclure à un effet général. Les générations sont coûteuses, non déterministes et réalisées avec un modèle fermé ; une étude plus large devrait répéter chaque prompt avec plusieurs graines et plusieurs catégories de scènes.

8 Analyse et discussion

8.1 Synthèse des résultats

Les expériences apportent une réponse nuancée à la problématique. L'expérience sur les paires démontre que la cohérence géométrique constitue bien un signal sensible aux déformations. De plus, la détection d'une image modifiée dans **73 %** des séquences, ainsi que les résultats obtenus sur **27 paires sur 33**, montrent que l'erreur de reprojection ne réagit pas de manière aléatoire. Lorsque deux vidéos représentent une même scène, la version générée est généralement plus difficile à expliquer à partir d'une profondeur, d'une caméra et d'un mouvement rigide cohérents.

Ce résultat confirme l'hypothèse de départ. Un modèle génératif peut produire des images visuellement réalistes tout en introduisant de légères incohérences géométriques au fil du temps. La forme d'un objet peut varier subtilement, un contour peut se déplacer indépendamment de la profondeur ou une région peut réapparaître avec un contenu différent.

En revanche, cette conclusion ne se vérifie plus lorsque les scènes ne sont plus appariées. Sur l'ensemble du jeu de données, les erreurs moyennes des vidéos réelles et générées sont trop proches pour définir un seuil de décision fiable. L'erreur photométrique ne mesure donc pas uniquement l'authenticité d'une vidéo ; elle dépend également de la capacité du pipeline à modéliser correctement la scène. Une vidéo réelle complexe peut ainsi obtenir un score élevé, tandis qu'une vidéo générée simple, peu dynamique et bien texturée peut être correctement reprojétée.

Il est donc possible de définir un seuil pertinent pour une scène donnée, par exemple lorsque plusieurs variantes d'un même prompt ou une vidéo réelle de référence sont disponibles. En revanche, ce seuil ne peut pas être généralisé à d'autres scènes. Ainsi, le principal résultat de ce stage n'est pas la conception d'un classifieur universel, **mais le développement d'un protocole de comparaison relative et d'un outil de diagnostic fondé sur l'analyse dense des incohérences géométriques.**

8.2 Limites de notre méthode

Une dépendance réduite aux estimateurs d'IA. Comme GeCo, notre pipeline s'appuie sur plusieurs estimations pour reconstruire la géométrie de la scène. Toutefois, cette dépendance est plus limitée. La pose de la caméra est par exemple estimée directement à partir des relations géométriques entre les images, sans recourir à un modèle d'intelligence artificielle. Surtout, notre métrique finale compare **l'image reconstruite aux intensités des pixels réellement observées dans la vidéo**. La décision repose donc sur une comparaison avec les données acquises, et non sur une comparaison entre plusieurs estimations successives comme GeCo, qui utilise le flot optique estimé comme référence. Notre approche réduit ainsi le nombre de couches d'estimation (*estimation layers*) susceptibles d'introduire ou de propager des erreurs jusqu'au score final.

Cependant, cette réduction du nombre d'estimateurs ne signifie pas que notre pipeline en est totalement indépendant. L'estimation de la profondeur ainsi que des paramètres intrinsèques de la caméra (*intrinsic*) repose toujours sur des modèles d'intelligence artificielle et constitue une

part importante de notre méthode. Il est donc nécessaire de rester prudent quant à l'influence que ces estimations peuvent avoir sur les résultats obtenus. C'est précisément cette question qu'étudie GeCo, en évaluant l'impact des erreurs introduites par ces estimateurs. GeCo reconnaît que sa métrique repose sur des estimateurs appris [6] (flow, profondeur et pose caméra), mais montre que leur impact reste limité. Les auteurs comparent les scores obtenus avec une géométrie prédite et une géométrie de référence, puis ajoutent artificiellement du bruit au flow et à la profondeur. Ils observent que GeCo reste robuste jusqu'à des niveaux de bruit largement supérieurs aux erreurs réelles des estimateurs. Ils en concluent que le « noise floor » introduit par cette couche d'estimation est bien inférieur aux incohérences géométriques produites par les modèles génératifs actuels, tout en reconnaissant que leur méthode hérite des limites de ces estimateurs.

Enfin, la méthode a été développée et évaluée sur des scènes rigides présentant peu de mouvements indépendants. Elle n'est donc pas directement applicable à des scènes complexes comportant des personnes, des objets articulés ou des interactions importantes.

8.3 Perspectives d'amélioration

Réduire la dépendance au mouvement. La première perspective consiste à enrichir les caractéristiques utilisées par le pipeline afin de limiter l'influence du mouvement de la scène sur l'erreur photométrique. Des informations telles que la norme et la distribution du flot optique, la proportion de zones occultées, les variations de profondeur ou encore les mouvements de rotation et de translation de la caméra pourraient être utilisées pour normaliser cette erreur. Plutôt que de définir un seuil unique pour toutes les vidéos, il serait alors possible d'adapter le score au type de scène ou à l'amplitude du mouvement observé.

Prendre en compte le contexte de la scène. Une seconde piste consiste à intégrer un modèle vision-langage (**GPT-4o**, **Claude 3.5 Sonnet** et **Qwen2.5-VL**) capable de décrire le contenu de la vidéo. Celui-ci pourrait fournir des informations sémantiques, comme la présence de surfaces réfléchissantes, d'objets articulés, de sources lumineuses mobiles ou de mouvements humains. L'objectif ne serait pas de remplacer l'analyse géométrique, mais de la compléter afin de mieux interpréter un score élevé et d'adapter le raisonnement au contexte observé. La combinaison d'une mesure physique explicable et d'une description sémantique constitue ainsi une piste prometteuse pour réduire la dépendance actuelle à la scène et pour aider notre classifieur à discriminer.

Exploiter davantage de contraintes géométriques. Enfin, une dernière perspective consiste à enrichir le raisonnement géométrique du pipeline. La reprojection ne représente qu'une seule des nombreuses contraintes géométriques présentes dans une vidéo. La cohérence des points de fuite, des lignes droites, des plans, des normales de surface, des ombres ou encore des trajectoires 3D pourrait fournir des signaux complémentaires.

9 Conclusion

Ce projet exploratoire avait pour objectif d'évaluer dans quelle mesure les incohérences géométriques pouvaient constituer un indice pertinent pour analyser des vidéos générées par intelligence artificielle. Les résultats obtenus montrent que **cette hypothèse est partiellement vérifiée**. Dans un cadre contrôlé, le pipeline distingue efficacement les vidéos réelles des vidéos générées et met en évidence les zones d'incohérence grâce à des cartes d'erreur interprétables. Il permet également d'évaluer l'influence de différentes stratégies de génération, notamment à travers l'étude des prompts.

Les expériences montrent toutefois que l'erreur de reprojection dépend fortement des caractéristiques de la scène, du mouvement, des occultations et des conditions d'acquisition. En conséquence,

elle ne constitue pas, à elle seule, un critère suffisant pour classer des vidéos arbitraires. L'intégration de GeCo n'a pas amélioré les performances de classification dans notre configuration, mais ses principes (distinction entre mouvement et structure, gestion des occultations et raisonnement temporel) ont permis de renforcer la robustesse et l'interprétabilité du pipeline.

Ainsi, la problématique posée trouve une réponse nuancée : les incohérences géométriques représentent bien un signal prometteur pour la détection de contenus générés, mais leur exploitation doit être complétée par d'autres indices afin d'aboutir à une méthode fiable et généralisable. Au-delà des résultats obtenus, ce stage m'a permis d'acquérir de solides compétences en vision par ordinateur 3D, notamment en géométrie projective, estimation de profondeur, estimation de pose caméra, flot optique et raisonnement géométrique appliqué à l'analyse de scènes. Il constitue ainsi une base solide pour de futurs travaux dans ce domaine.

Références

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza et al., “Generative Adversarial Nets”, *Advances in Neural Information Processing Systems*, 2014. <https://arxiv.org/abs/1406.2661>
- [2] L. Yang, Z. Zhang, Y. Song et al., “Diffusion Models : A Comprehensive Survey of Methods and Applications”, 2022. <https://arxiv.org/abs/2209.00796>
- [3] S.-Y. Wang, O. Wang, R. Zhang, A. Owens et A. A. Efros, “CNN-generated images are surprisingly easy to spot... for now”, *Proceedings of CVPR*, 2020. <https://arxiv.org/abs/2003.01826>
- [4] N. Krawetz, “A Picture’s Worth : Digital Image Analysis and Forensics”, *Black Hat USA*, 2007. <https://blackhat.com/presentations/bh-usa-07/Krawetz/Whitepaper/bh-usa-07-krawetz-WP.pdf>
- [5] “Three-dimensional reconstruction for non-destructive testing and aerospace applications”, *NDT & E International*, 2025. <https://www.sciencedirect.com/science/article/pii/S1051200425005573>
- [6] L. Gu, J. Hur et al., “GeCo : Geometry Consistency for Generated Videos”, 2025. <https://arxiv.org/abs/2512.22274>
- [7] P. Monasse, “Géométrie de la vision : stéréovision et géométrie multi-vue”, supports de cours, École des Ponts ParisTech. <https://imagine.enpc.fr/~monasse/Stereo/>
- [8] E. Prados et O. Faugeras, “Shape from Shading”, chapitre de synthèse, INRIA, 2006. <https://perception.inrialpes.fr/Publications/2006/PF06a/chapter-prados-faugeras.pdf>
- [9] “Depth Anything 3”, page du projet et documentation du modèle. <https://depth-anything-3.github.io/>
- [10] M. Oquab, T. Darcet, T. Moutakanni et al., “DINOv2 : Learning Robust Visual Features without Supervision”, 2023. <https://arxiv.org/abs/2304.07193>
- [11] Y. Veicht et al., “GeoCalib : Single-image Camera Calibration with Geometric Optimization”, 2024. <https://arxiv.org/abs/2409.06704>
- [12] J. Wang et al., “VGGT : Visual Geometry Grounded Transformer”, 2025. <https://arxiv.org/abs/2503.11651>
- [13] OpenAI, “Video generation models as world simulators”, rapport technique Sora, 2024. <https://openai.com/index/video-generation-models-as-world-simulators/>
- [14] D. Reizenstein, R. Shapovalov, P. Henzler et al., “Common Objects in 3D : Large-Scale Learning and Evaluation of Real-life 3D Category Reconstruction”, 2021. <https://arxiv.org/abs/2109.00512>
- [15] Luma AI, “Dream Machine — Ray video models”, documentation du service. <https://lumalabs.ai/dream-machine>
- [16] F. L. Bookstein, “Principal Warps : Thin-Plate Splines and the Decomposition of Deformations”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1989. <https://doi.org/10.1109/34.24792>
- [17] S. K. Lam, A. Pitrou et S. Seibert, “Numba : A LLVM-based Python JIT Compiler”, *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015. <https://numba.pydata.org/>
- [18] P. Geurts, D. Ernst et L. Wehenkel, “Extremely Randomized Trees”, *Machine Learning*, vol. 63, 2006. <https://doi.org/10.1007/s10994-006-6226-1>
- [19] *Le Monde*, dossier “Deepfakes”. <https://www.lemonde.fr/deepfakes/>
- [20] J. L. Schönberger et J.-M. Frahm, “Structure-from-Motion Revisited”, *Proceedings of CVPR*, 2016. <https://colmap.github.io/>
- [21] OpenCalib, “CalibAnything : Zero-training Camera Calibration Method for Any Camera”, dépôt logiciel. <https://github.com/OpenCalib/CalibAnything>
- [22] A. J. Davison, I. D. Reid, N. D. Molton et O. Stasse, “MonoSLAM : Real-Time Single Camera SLAM”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2007. https://scholar.google.com/citations?view_op=view_citation&hl=en&user=cXQciMEAAAAJ&citation_for_view=cXQciMEAAAAJ:0CzhzZyukY4C

A Annexes

A.1 Distributions des estimations sur la séquence hydrant

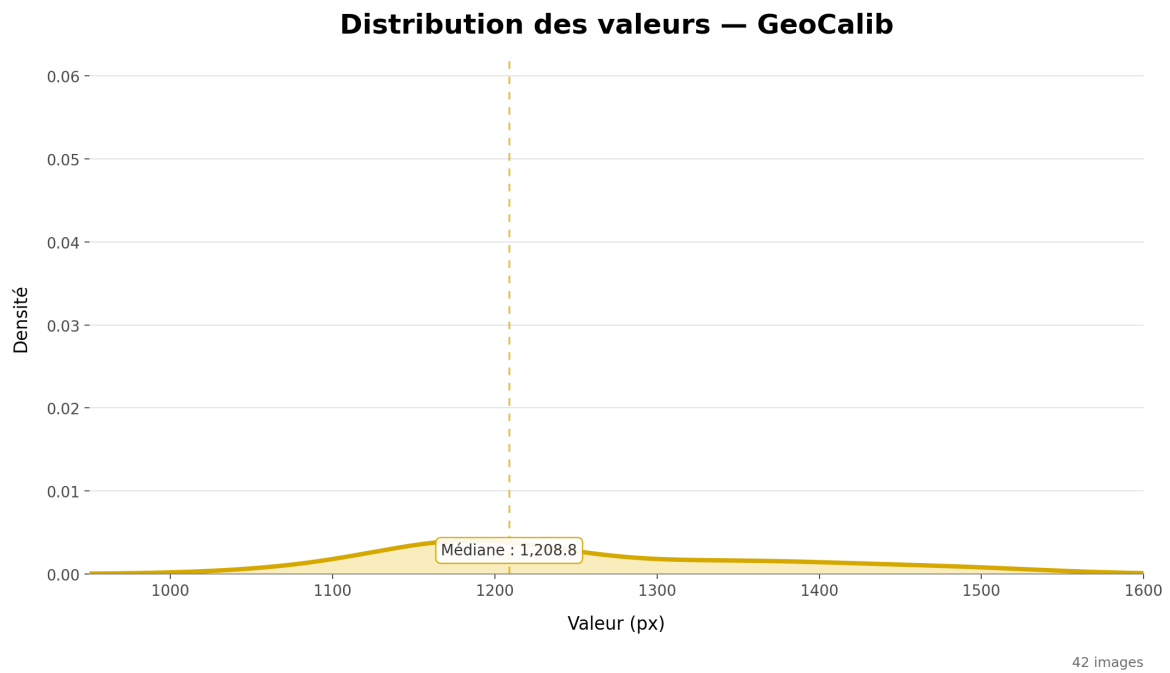


Figure 13 – Distribution des paramètres intrinsèques estimés par GeoCalib sur la séquence hydrant.

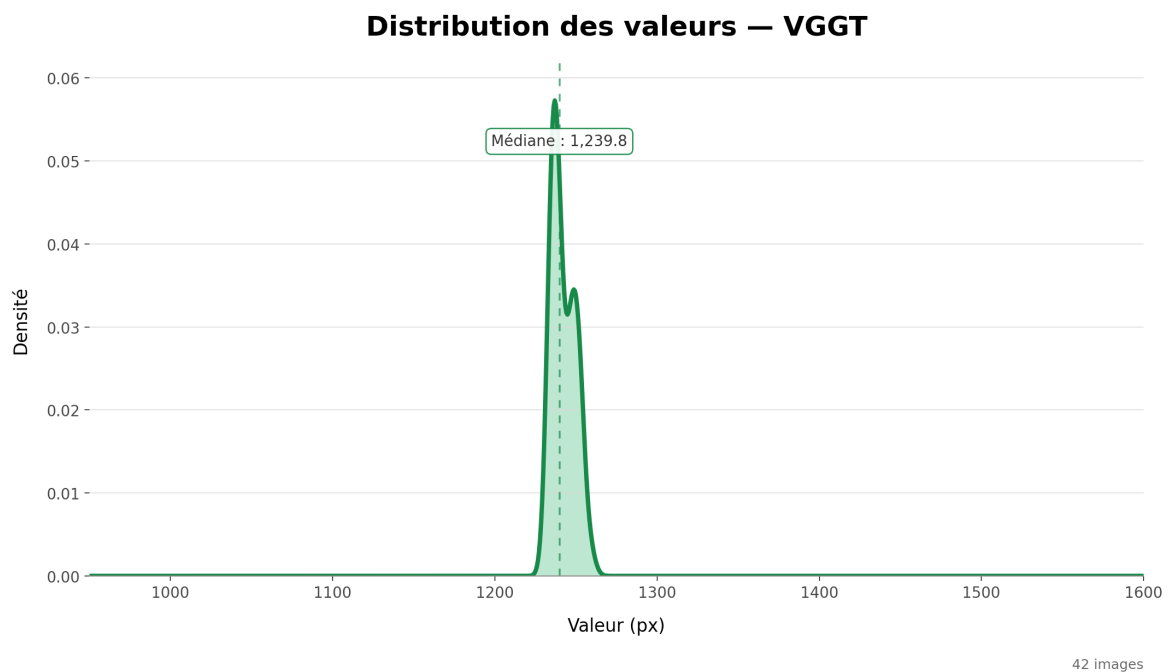


Figure 14 – Distribution des paramètres intrinsèques estimés par VGGT sur la séquence hydrant.

A.2 Résultats par paire

Catégorie	Séquence	Méthode d'intrinsèques	Score réel	Score généré	Variation
object_centric	34_1479_4753_teddybear	vggt_mean	3,776	7,880	52,1 %
object_centric	50_2928_8645_suitcase	vggt_mean	3,227	8,925	63,8%
object_centric	69_5465_12831_bowl	vggt_mean	3,437	9,404	63,4%
object_centric	70_5792_13401_bowl	vggt_mean	7,973	12,923	38,3%
object_centric	107_12753_23606_mouse	vggt_mean	5,911	8,540	30,8%
object_centric	119_13962_28926_book	vggt_mean	6,972	10,062	30,7%
object_centric	123_14363_28981_ball	vggt_mean	7,849	11,927	34,2%
object_centric	167_18184_34441_hydrant	vggt_mean	6,133	7,162	14,4%
object_centric	187_20215_38541_teddybear	vggt_mean	7,276	12,318	40,9%
object_centric	189_20393_38136_apple	vggt_mean	7,532	10,388	27,5%
object_centric	190_20494_39385_toytruck	vggt_mean	4,719	10,429	54,8%
object_centric	195_20989_41543_remote	vggt_mean	7,781	9,818	20,7%
object_centric	240_25394_51994_toytrain	vggt_mean	6,091	6,983	12,8%
object_centric	245_26182_52130_skateboard	vggt_mean	6,733	9,740	30,9%
object_centric	247_26441_50907_plant	vggt_mean	4,130	10,044	58,9%
object_centric	247_26469_51778_book	vggt_mean	4,886	6,687	26,9%
object_centric	346_36113_66551_toytruck	vggt_mean	6,604	12,317	46,4%
object_centric	350_36761_68623_remote	vggt_mean	6,188	11,449	45,9%
object_centric	366_39266_76077_skateboard	vggt_mean	3,200	6,911	53,7%
object_centric	374_41862_83720_vase	vggt_mean	7,599	4,399	-72,7%
object_centric	374_42005_84358_plant	vggt_mean	5,796	9,527	39,2%
object_centric	374_42196_84367_orange	vggt_mean	5,375	9,241	41,8%
object_centric	375_42693_85518_ball	vggt_mean	5,465	6,304	13,3%
object_centric	377_43416_86289_mouse	vggt_mean	4,008	5,248	23,6%
object_centric	380_44863_89631_vase	vggt_mean	5,251	7,145	26,5%
object_centric	385_45386_90752_orange	vggt_mean	6,447	8,220	21,6%
object_centric	391_47032_93657_donut	vggt_mean	3,488	5,920	41,1%
object_centric	399_51323_100753_toytrain	vggt_mean	4,898	6,231	21,4%
object_centric	403_52964_103416_donut	vggt_mean	7,644	7,775	1,7%
object_centric	410_55734_107452_suitcase	vggt_mean	6,584	7,363	10,6%
object_centric	411_56064_108483_hydrant	vggt_mean	3,859	7,625	49,4%
object_centric	415_57112_110099_bench	vggt_mean	7,359	12,058	39,0%
object_centric	415_57121_110109_bench	vggt_mean	7,107	9,123	22,1%
object_centric	426_59761_116348_toaster	vggt_mean	7,634	10,491	27,2 %
object_centric	410_55734_107452_2_suitcase	vggt_mean	4,877	5,943	17,9 %

Les deux dernières lignes complètent les 33 paires principales par des résultats supplémentaires conservés à titre documentaire.

La géométrie permet de percevoir la structure de la scène; elle enrichit ainsi les approches d'apprentissage profond.